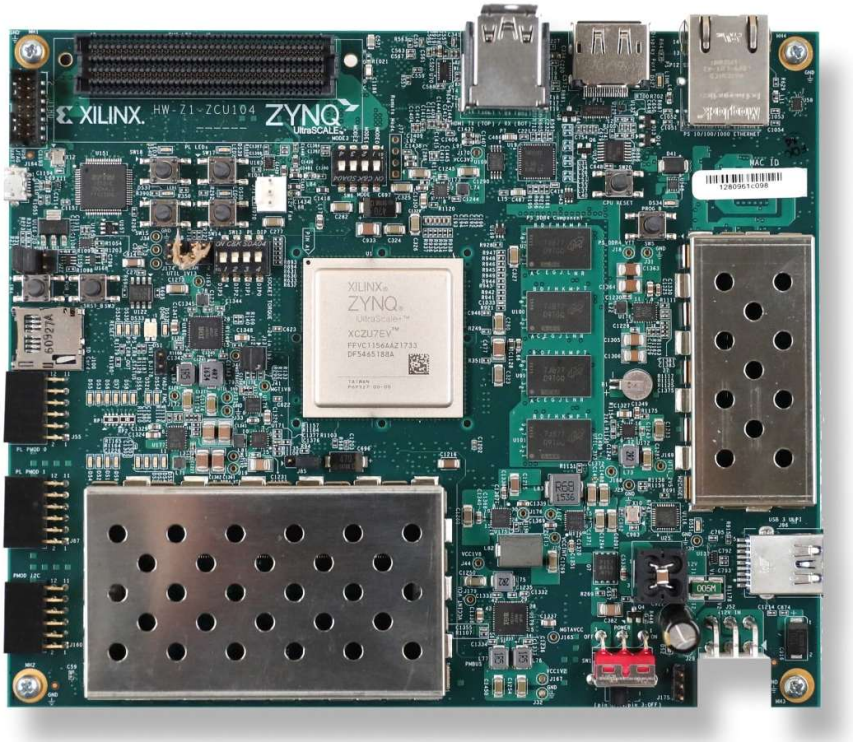
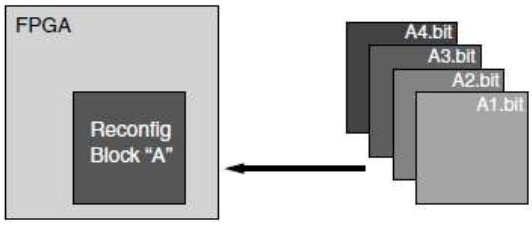
Embedded System Design Lab – Jeju National University

Methodology and Workflow for Partial Reconfiguration in an FPGA SoC System

Xilinx Dynamic Function eXchange Tutorial





Basic Premise of Partial Reconfiguration

TRAN VAN DINH

2026-05-23

https://github.com/vandinhtranengg/zcu104-dfx-calculator-demo

Contents

1. Introduction

This guide explains the methodology and workflow for designing, implementing, and validating partial reconfiguration in an FPGA SoC system. The practical demonstration uses the Xilinx ZCU104 board, Vivado, Vitis HLS, Vitis IDE, and the Xilinx Dynamic Function eXchange (DFX) flow.

The example reconfigurable hardware function is intentionally simple: one reconfigurable module performs addition and another performs subtraction. The purpose is not the calculator itself; the purpose is to teach the complete DFX workflow: define a stable reconfigurable partition boundary, create interchangeable reconfigurable modules, protect the AXI interface during reconfiguration, generate partial bitstreams, and control the design from bare-metal software.

There are two common ways to implement a DFX project in Vivado. The first method is the Block Design Container (BDC) and DFX Wizard flow, which is recommended and mainly used in this tutorial because it is easier for less-experienced users and works well with IP Integrator block designs. The second method is a manual flow using Tcl commands. The manual flow is useful for advanced scripting and for understanding the internal mechanism, but it is more difficult to debug for beginners.

Main purpose of this tutorial: *To share a reusable design methodology for partial reconfiguration in FPGA SoC systems. Lab members can later replace the ADD/SUB calculator modules with image filters, sequence-alignment accelerators, AI preprocessing blocks, protocol accelerators, or other hardware functions while keeping the same DFX workflow.*

2. Project Overview Objective

Implement a minimal but complete Dynamic Function eXchange demonstration on the ZCU104 platform. The Processing System controls the Programmable Logic, loads partial bitstreams at runtime through the PCAP configuration path, and accesses the active hardware module through an `AXI4-Lite` register interface.

Features

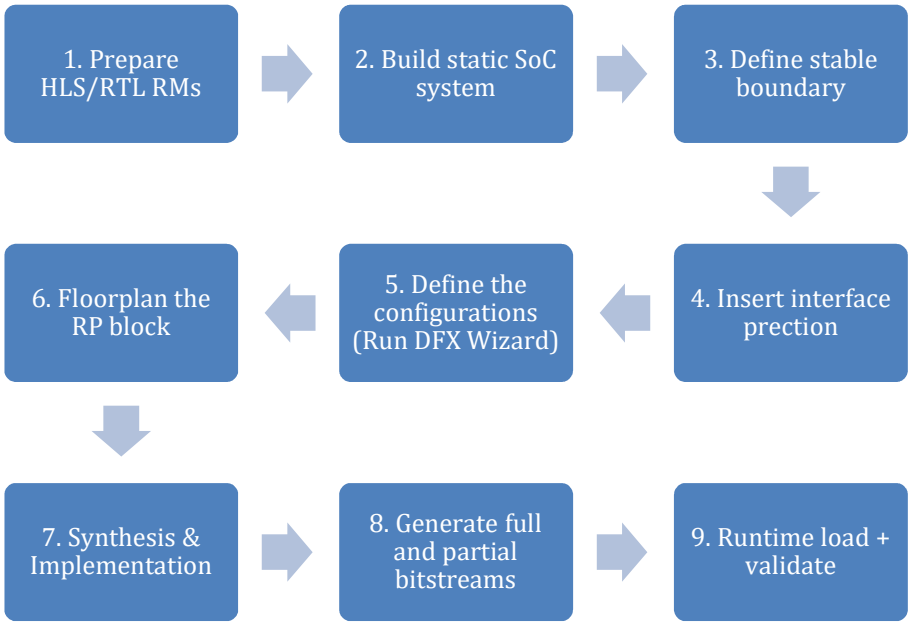
- One Static SoC design containing the PS, AXI interconnect, reset logic, and DFX AXI Shutdown Manager.
- One Reconfigurable Partition named `calculator_rp`.
- Two Reconfigurable Modules: ADD and SUB.
- Both RMs use the same AXI4-Lite register map, clock, reset, and boundary ports.
- Configuration method: XilFPGA driver through PS PCAP path
- Bare-metal runtime sequence: shutdown AXI interface, load partial bitstream, reconnect AXI interface, then access the active IP.
- Validation using result value and `module_id` value.

Tools and Versions

Tool / Board	Version / Target	Purpose
Board	ZCU104	FPGA SoC target platform
Vitis HLS	2022.1	Create ADD and SUB AXI4-Lite HLS IPs
Vivado	2022.1	Create BDC/DFX design, pblock, full and partial bitstreams
Vitis IDE	2022.1	Bare-metal application and BSP
SD card	FAT32	Store `ADD.BIT` and `SUB.BIT` for runtime loading

3. Dynamic Function eXchange Methodology

DFX separates the FPGA design into two conceptual areas: a static region and one or more reconfigurable partitions. The static region remains active while a reconfigurable partition is replaced by a partial bitstream. Correct DFX design is therefore mostly an interface, floor-planning and runtime-control discipline.



GENERAL DFX DESIGN AND IMPLEMENTATION WORKFLOW

Methodological focus: the ADD/SUB calculator in this tutorial is only a minimal example. The same workflow could apply to other particular design such as filters, accelerator, protocol blocks or algorithm-specific hardware modules.

Core Design Rules

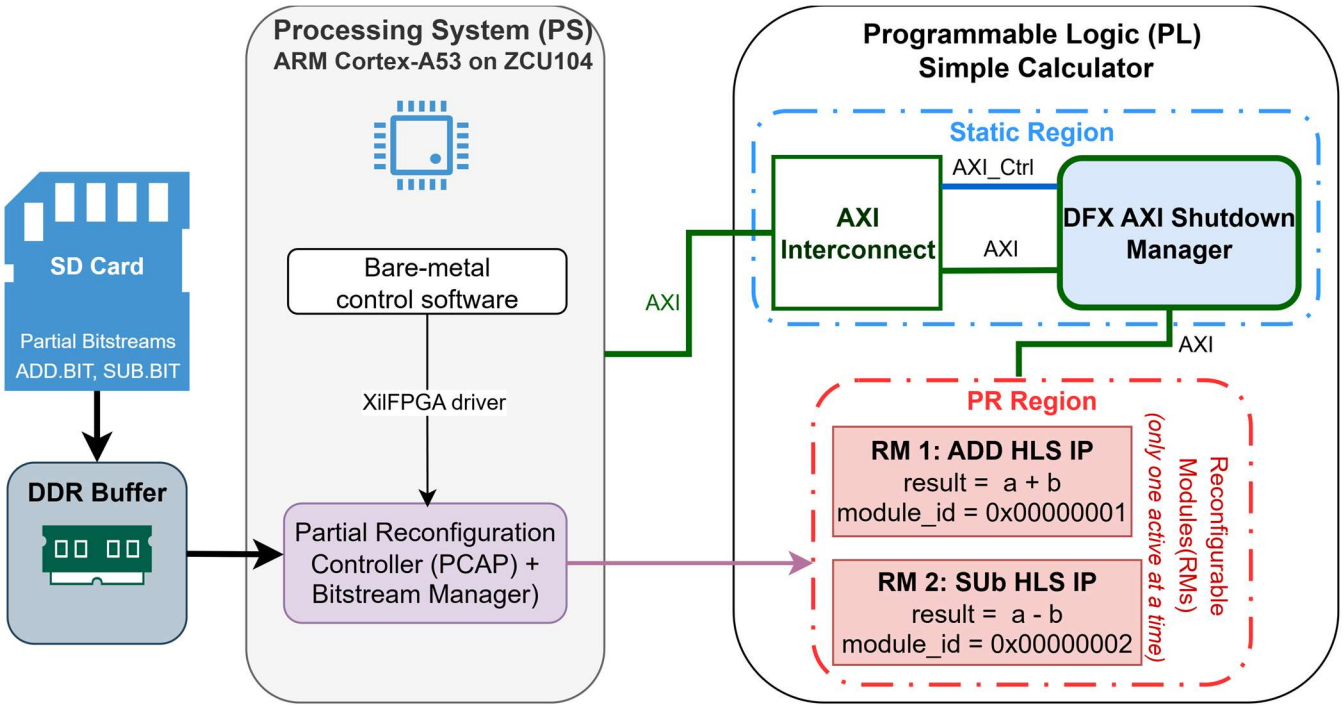
- The RP boundary must remain stable. All RMs for the same RP must expose identical external ports.
- Clock, reset, AXI protocol, interrupt, and other boundary signals must be consistent across all RMs.
- Static logic must not be placed inside the RP pblock.
- The RP must be assigned a valid pblock region.
- AXI transactions to the RP must be stopped or safely terminated during partial reconfiguration.
- Software must not access RP registers while the RP is being reconfigured.

Decoupler / Shutdown Manager selection guide

RP interface situation	Recommended protection	Reason
AXI4-Lite slave only	DFX AXI Shutdown Manager	Safely blocks or terminates AXI register accesses during PR
AXI4 memory-mapped interface	DFX AXI Shutdown Manager	Prevents incomplete memory-mapped transactions from hanging
AXI4-Stream interface	DFX Decoupler or stream-safe isolation	Holds stream handshake/data signals in a safe state
Simple output pins / status signals	DFX Decoupler	Drives known values while the RP is reconfigured
Mixed AXI + custom pins	Shutdown Manager for AXI + Decoupler for custom pins	Each boundary type needs appropriate isolation

4. Reference SoC Architecture

The reference architecture uses the PS as the controller and the PL as the hardware fabric. The PS accesses the HLS IP through AXI4-Lite and loads partial bitstreams using the PS-side PCAP configuration path.



REFERENCE FPGA SOC DFX ARCHITECTURE

Static Region

- Zynq UltraScale+ MPSoC Processing System
- AXI Interconnect
- Processor System Reset block
- DFX AXI Shutdown Manager
- Clocking and reset routing

Reconfigurable Region

- Reconfigurable Partition: `calculator_rp`
- RM1: `rp_calculator_add`
- RM2: `rp_calculator_sub`
- Same AXI4-Lite register map and same interface boundary

5. Section 1: HLS IP for the PR-Block

In this tutorial, this tutorial assumes familiarity with HLS IP creation; if not, please refer this guide https://github.com/vandinhtranengg/zybo_memcopy_accel_interrupt first.

Prepare hardware functions that may be loaded into the RP. For this tutorial, the two modules are `rp_calculator_add` and `rp_calculator_sub`. Both HLS IPs code is attached through GitHub. Also, the other source files, exported IP repositories, Vivado constraints, Vitis application, and documentation are available in this GitHub repository so readers can reproduce the project. Clone or download it as needed.

rp_calculator_add.cpp

```
#include <ap_int.h>

void rp_calculator_add(
    ap_uint<32> a,
    ap_uint<32> b,
    ap_uint<32> *result,
    ap_uint<32> *module_id
) {
#pragma HLS INTERFACE s_axilite port=a         bundle=CTRL
#pragma HLS INTERFACE s_axilite port=b         bundle=CTRL
#pragma HLS INTERFACE s_axilite port=result    bundle=CTRL
#pragma HLS INTERFACE s_axilite port=module_id bundle=CTRL
#pragma HLS INTERFACE s_axilite port=return    bundle=CTRL

    *result = a + b;
    *module_id = 0x00000001;
}

```

rp_calculator_sub.cpp

```
#include <ap_int.h>

void rp_calculator_sub(
    ap_uint<32> a,
    ap_uint<32> b,
    ap_uint<32> *result,
    ap_uint<32> *module_id
) {
#pragma HLS INTERFACE s_axilite port=a         bundle=CTRL
#pragma HLS INTERFACE s_axilite port=b         bundle=CTRL
#pragma HLS INTERFACE s_axilite port=result    bundle=CTRL
#pragma HLS INTERFACE s_axilite port=module_id bundle=CTRL
#pragma HLS INTERFACE s_axilite port=return    bundle=CTRL

    *result = a - b;
    *module_id = 0x00000002;
}

```

The HLS modules are deliberately simple so that the DFX mechanism is easy to debug. The ADD and SUB modules use the same top-level function signature and the same AXI4-Lite interface pragmas. This is essential because Vivado DFX requires all modules assigned to the same RP to have compatible boundary interfaces.

One important thing to note after synthesis is the AXI-Lite Register Map. This table defines the memory-mapped AXI-Lite control interface that allows the PS software to start the HLS IP, send input data, read output data, check status, and confirm which DFX module is currently loaded.

S_AXILITE Registers

Interface	Register	Offset	Width	Access	Description	Bit Fields
s_axi_CTRL	CTRL	0x00	32	RW	Control signals	0=AP_START 1=AP_DONE 2=AP_IDLE 3=AP_READY 7=AUTO_RESTART 9=INTERRUPT
s_axi_CTRL	GIER	0x04	32	RW	Global Interrupt Enable Register	0=Enable
s_axi_CTRL	IP_IER	0x08	32	RW	IP Interrupt Enable Register	0=CHAN0_INT_EN 1=CHAN1_INT_EN
s_axi_CTRL	IP_ISR	0x0c	32	RW	IP Interrupt Status Register	0=CHAN0_INT_ST 1=CHAN1_INT_ST
s_axi_CTRL	a	0x10	32	W	Data signal of a	
s_axi_CTRL	b	0x18	32	W	Data signal of b	
s_axi_CTRL	result	0x20	32	R	Data signal of result	
s_axi_CTRL	result_ctrl	0x24	32	R	Control signal of result	0=result_ap_vld
s_axi_CTRL	module_id	0x30	32	R	Data signal of module_id	
s_axi_CTRL	module_id_ctrl	0x34	32	R	Control signal of module_id	0=module_id_ap_vld

HLS IP AXI4-Lite Register Map

Methodology note: *For a real project, the internal algorithm can be different for every RM, but the boundary must stay the same. For example, RM1 can be a Sobel filter and RM2 can be a Gaussian filter only if both expose the same external interface.*

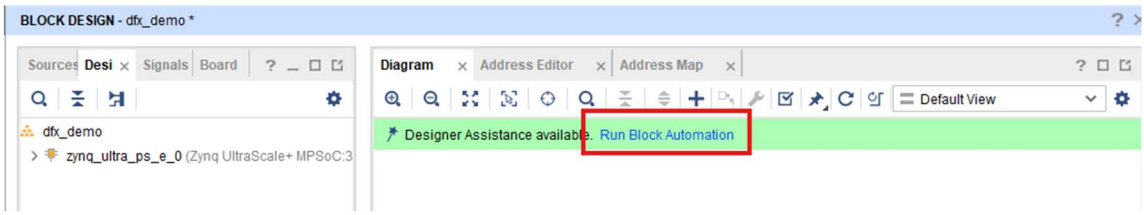
6. Section 2: Vivado Hardware Flow

Create the Vivado Project and Import HLS IPs

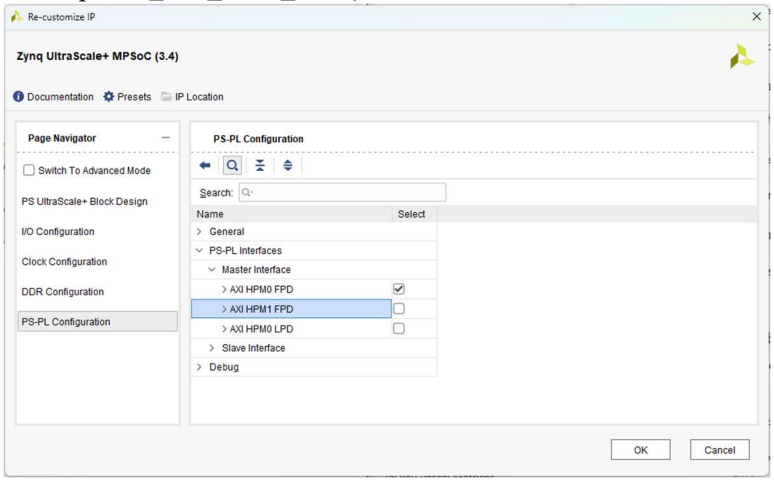
1. Create a Vivado RTL project for the ZCU104 board or the corresponding Zynq UltraScale+ device.
2. Add the exported HLS IP repositories for `rp_calculator_add` and `rp_calculator_sub`.

Create the Base Block Design

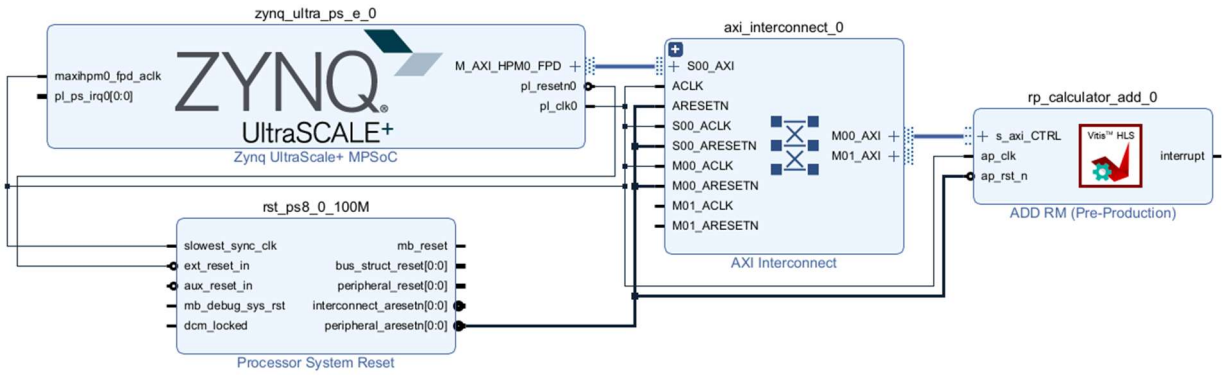
3. Create an IP Integrator block design.
4. Add the Zynq UltraScale+ MPSoC and Run Block Automation to enable at least `M_AXI_HPM0_FPD`, `pl_clk0`, and `pl_resetn0` for PS-to-PL control



5. Disable the unused interface port `M_AXI_HPM1_FPD` (double click the PS instance → PS-PL Configuration)

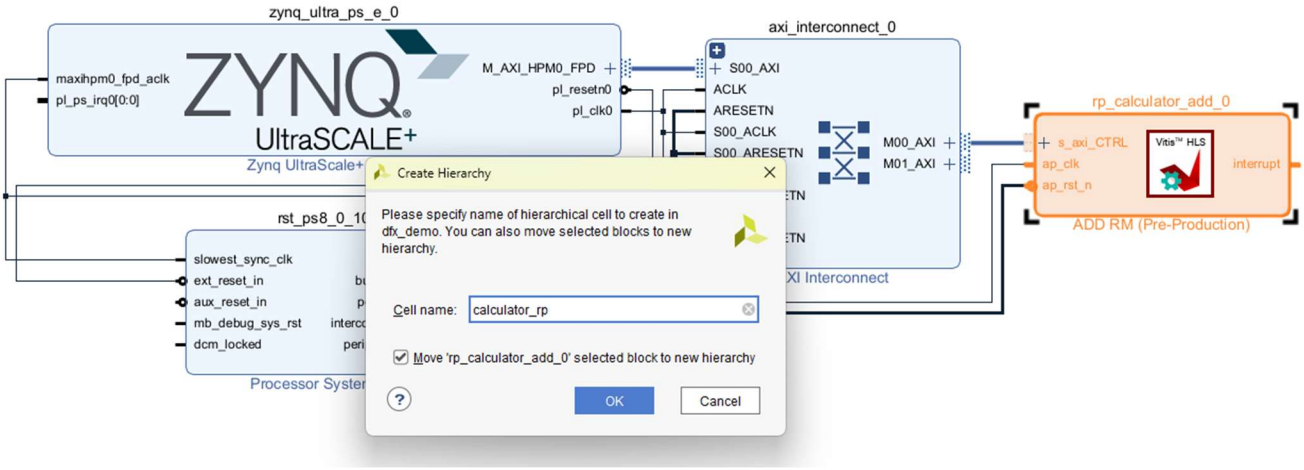


6. Add AXI Interconnect; double click on it to customize, change to set Number of Slave Interfaces to 1, Number of Master Interfaces to 1, then select OK to return the canvas.
7. Add ADD RM (`rp_calculator_add`).
8. Click Run Connection Automation we will have the base design as shown in the figure below. There is Zynq UltraScale+ MPSoC processing block, a reset controller, and an ADD RM connected via an AXI Interconnect. The `rp_calculator_add_0` will be in a DFX-enabled Block Design Container that can be reconfigured by the user.

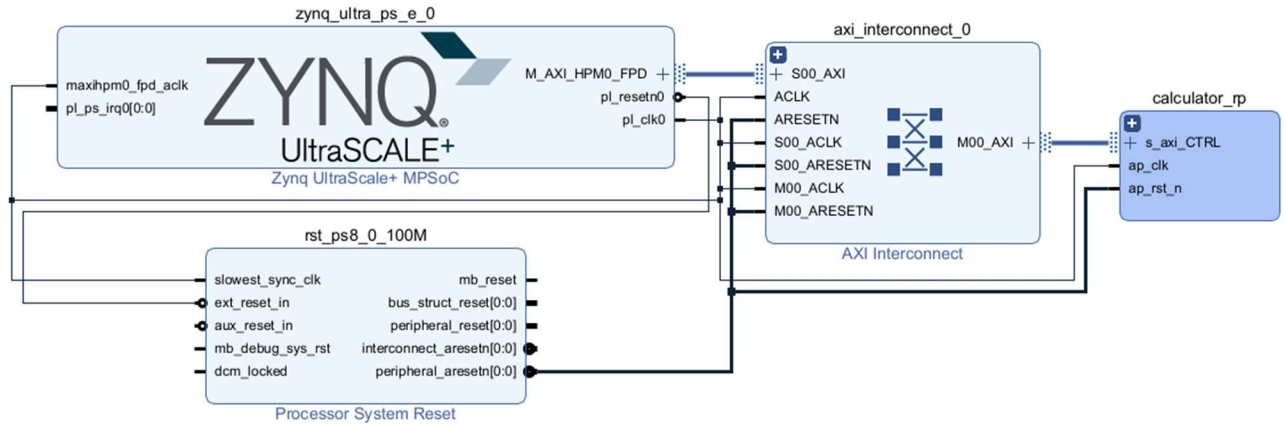


Create Block Design Container

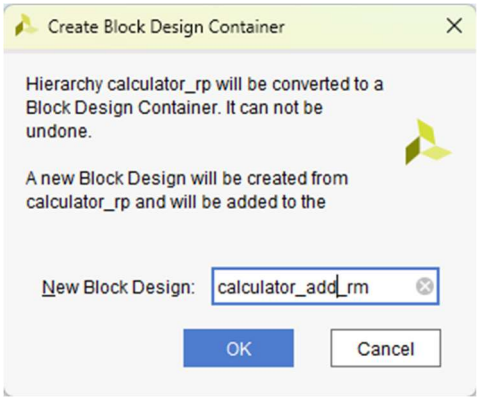
9. In the block design canvas, select `rp_calculator_add_0`, then right-click and select Create Hierarchy. Name the hierarchy `calculator_rp`. Press OK to return to the canvas.



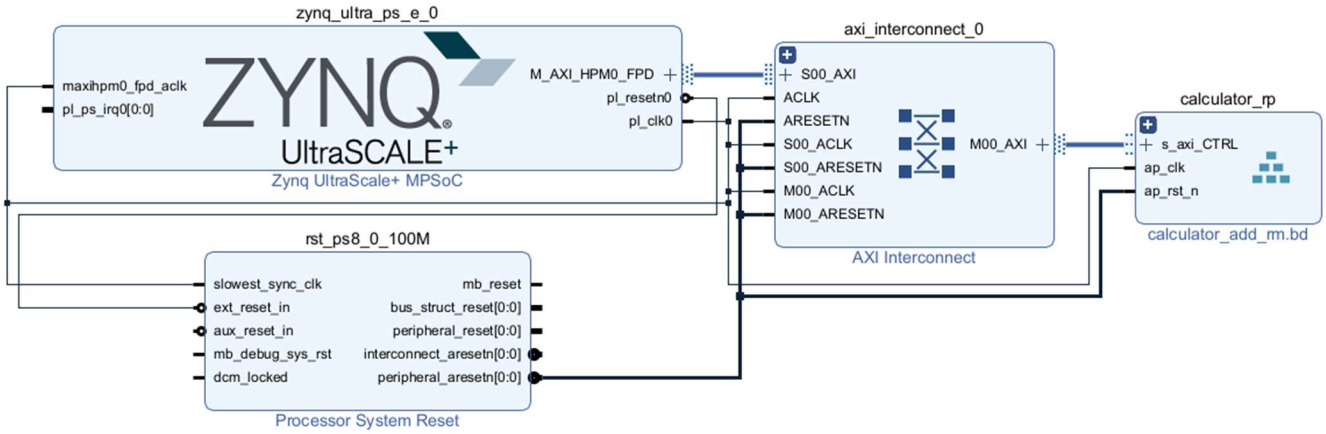
The block design will now look like this:



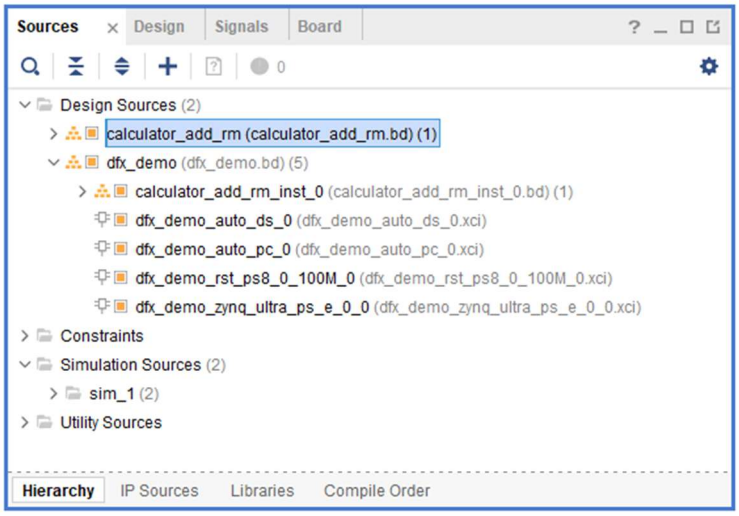
10. Validate the block design (use the ☑ icon) and then save the design.
11. Create the Block Design Container by right-clicking on `calculator_rp`, selecting Create Block Design Container, and giving it the name `calculator_add_rm`. Click OK to complete the process.



This will convert the hierarchical instance into a Block Design Container. This block will be labeled `calculator_add_rm.bd` and will have an icon that looks like a pyramid of six rectangles.



Also, you will see this new block design has been added to the project, visible in the sources view.

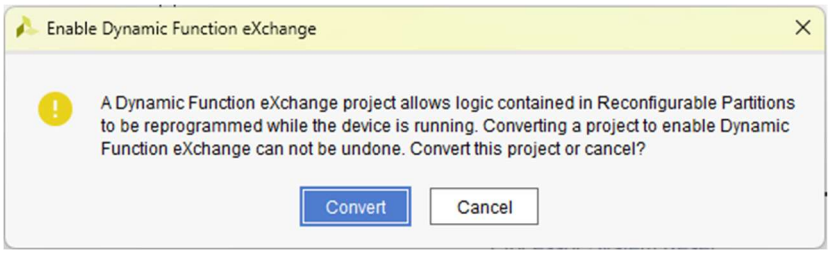


12. **Validate** then **Save** the design

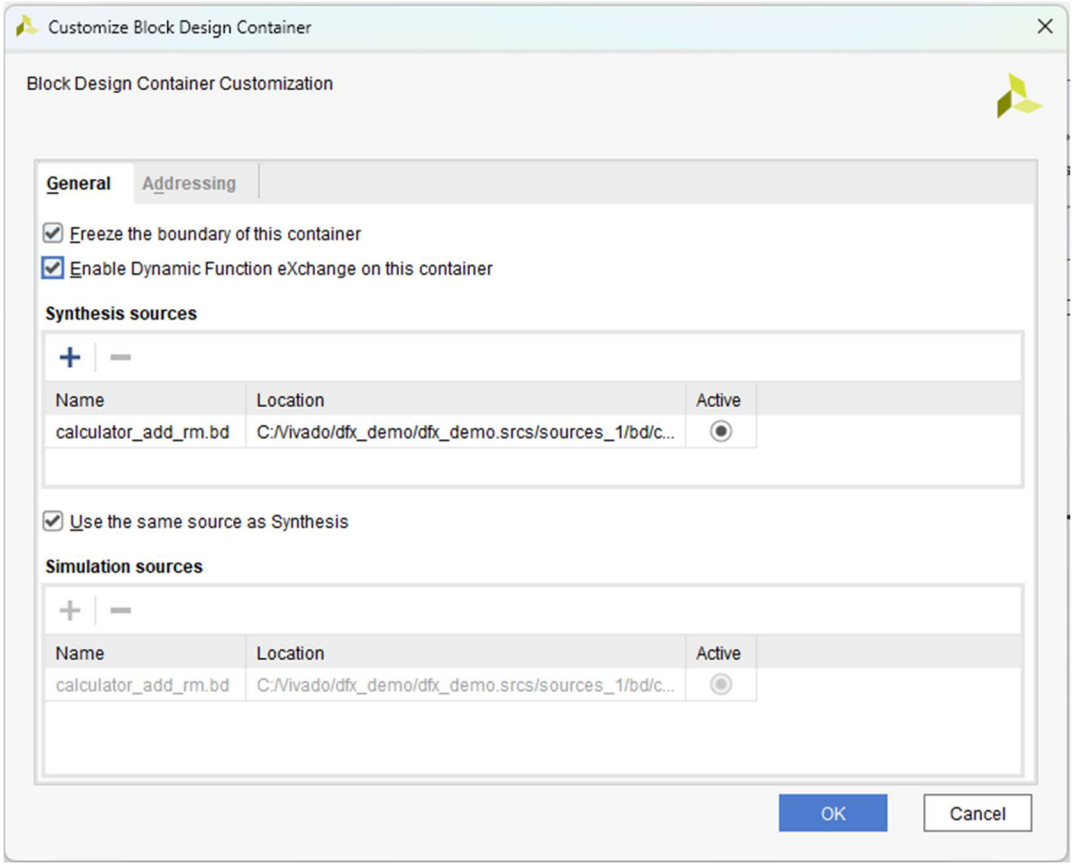
At this point the design is still a standard IP Integrator project, except with two block designs instead of one. The Block Design Container feature in IP Integrator allows you to add multiple variants of the `calculator_rp` block design through multiple design revisions; in other words, it helps us to do the partial reconfiguration which could not do if using the standard IP instance directly.

Enable Dynamic Function eXchange

13. In the Vivado IDE, enable the DFX flow by selecting **Tools** > **Enable Dynamic Function eXchange**. Select Convert in the dialog box that opens.

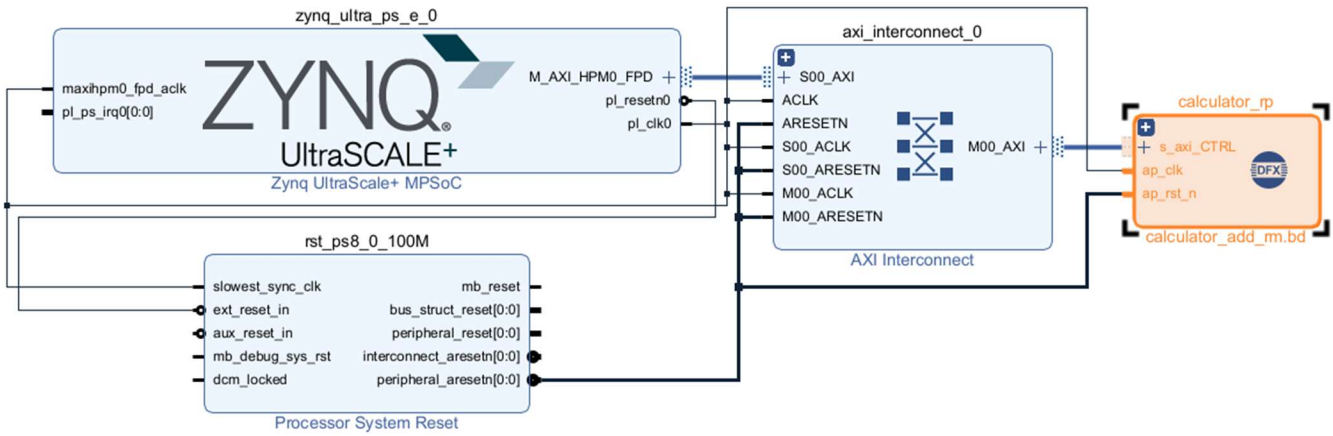


14. Back in the `dfx_demo` block design, **double-click** on the `calculator_rp` instance to edit the Block Design Container.
15. Under the **General** tab, check the **Enable Dynamic Function eXchange** on this container checkbox. This turns the BDC into a Reconfigurable Partition.
    Also check the **Freeze the boundary of this container** checkbox.



Until the boundary is frozen you can pass parameters through the hierarchy into the Block Design Container. However, in order to implement the DFX design the boundary must be frozen.

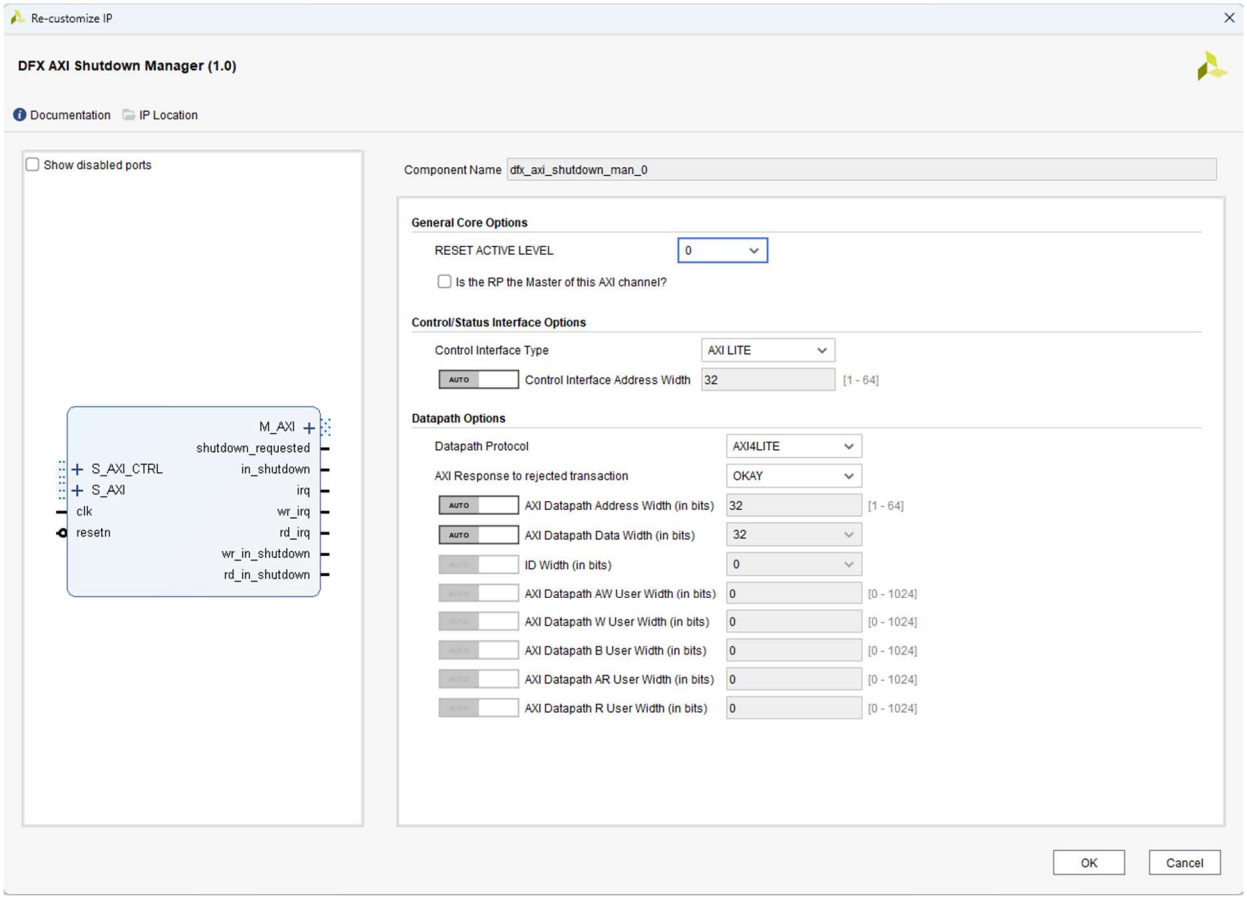
16. Click **OK** to save the changes and return to the `dfx_block` design.
17. **Validate** and then **Save** the design.
    The `calculator_rp` block container instance will now have a DFX label.



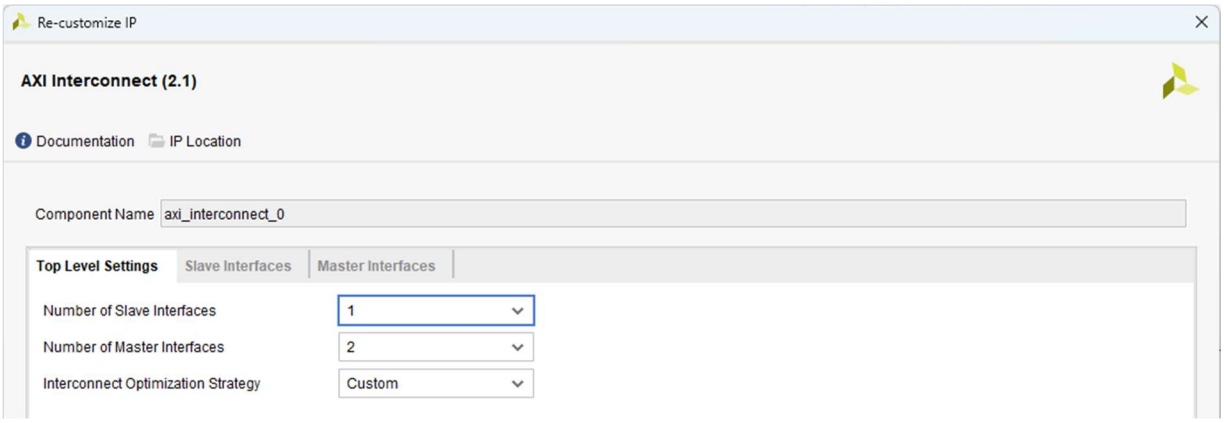
Add DFX AXI Shutdown Manager

The shutdown manager protects the static AXI system from hanging when the RP is unavailable during reconfiguration. This is critical for AXI4-Lite controlled HLS IPs.

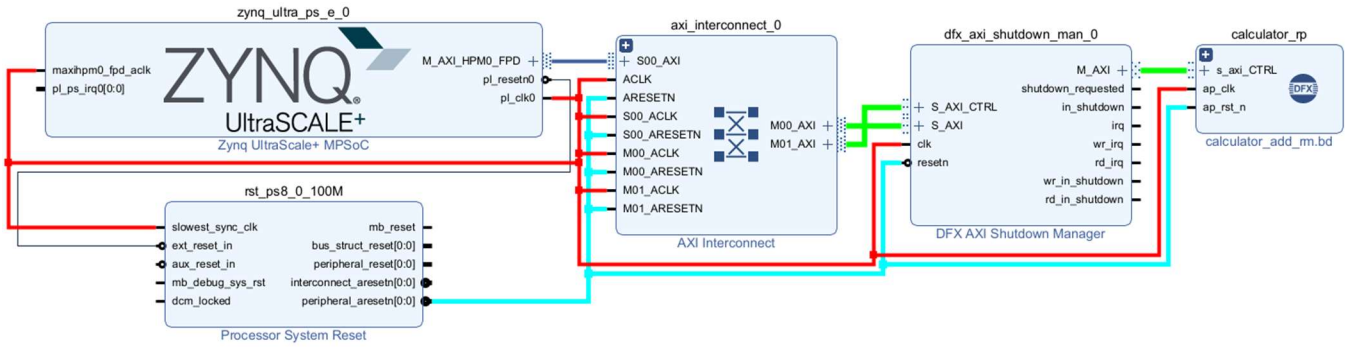
18. From the canvas click to + icon and using the search field add a **DFX AXI Shutdown Manager IP** core to the design.
19. Double-click to customize the core.
    - Deselect the **Is the RP the Master of this AXI channel?** checkbox.
    - Set the **Control Interface Type to AXI LITE**.
    - Change the **Datapath Protocol to AXI4LITE**.
    - Set the **AXI Response to rejected transaction** option to **OKAY**.



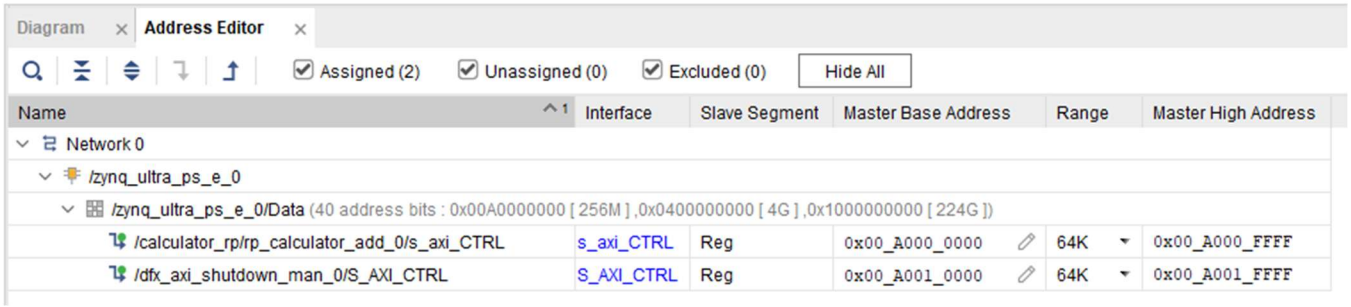
20. Double click on `axi_interconnect_0` to customize, change to set the **Number of Master Interfaces** to **2**, then select **OK** to return to the canvas.



21. Modify the interfaces, clocks and resets to/from the shutdown manager, calculator, and `axi_interconnect_0` to match the following diagram.



22. Under the Address Editor tab assign addresses using the Assign All Icon
You will end up with the following in the Address Editor

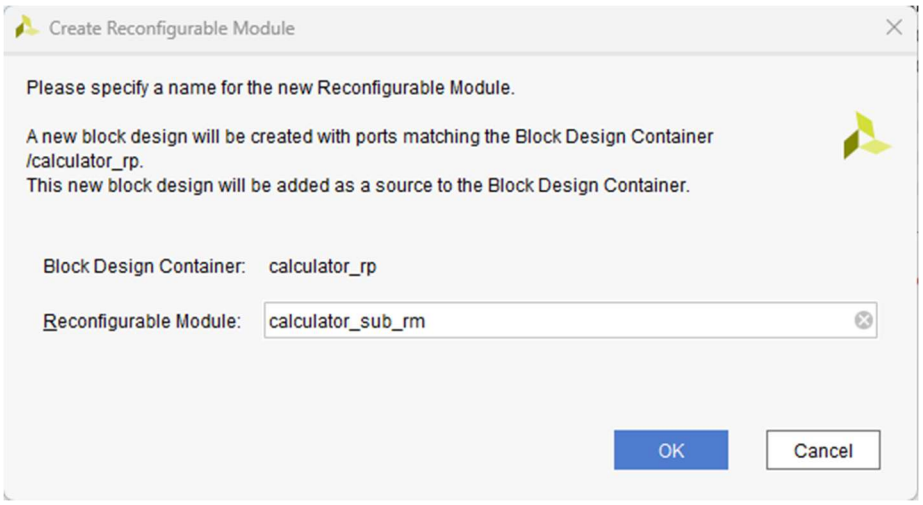


23. **Validate** and then **save** the design.

Add a new Reconfigurable Module

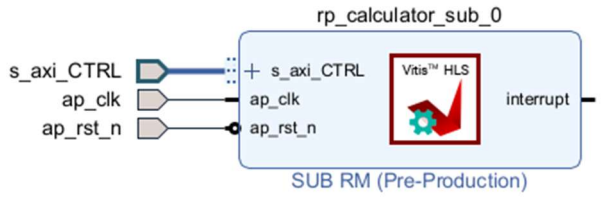
You will now create a second Reconfigurable Module - the Subtractor `rp_calculator_sub`.

24. Right-click on the `calculator_rp` instance and select **Create Reconfigurable Module**. Use `calculator_sub_rm` for the reconfigurable module name, then press **OK**.

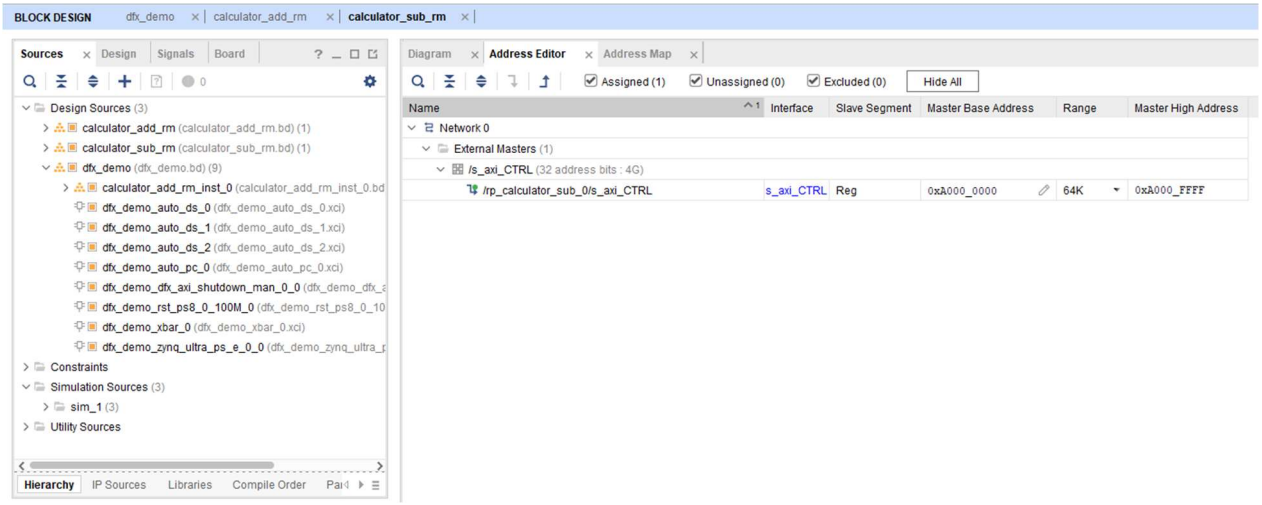


A new block design will be created with the interface pins defined by the initial Reconfigurable Module. The interface pins between all RMs for a given RP must be identical, even if some of the pins are not used by each RM. Unused ports can be tied off.

25. With the `calculator_sub_rm` block design open, add an HLS SUB RM IP (`rp_calculator_sub`) instance.
26. Connect the `ap_clk`, `ap_rst_n`, `s_axi_CTRL` port to the corresponding pins on the instance.



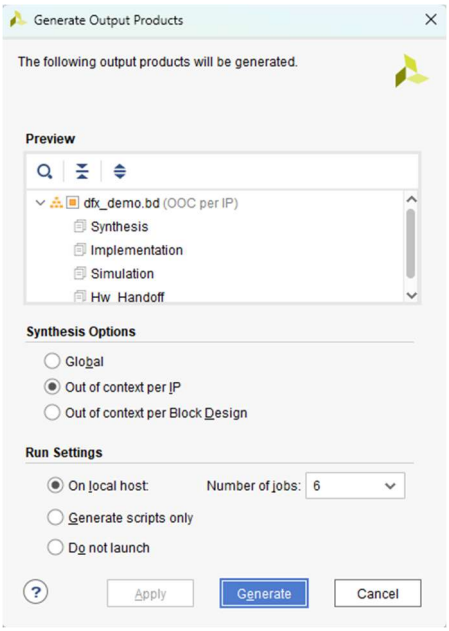
27. Under the Address Editor tab, assign the same address with rp_calculator_add 0xA000_0000



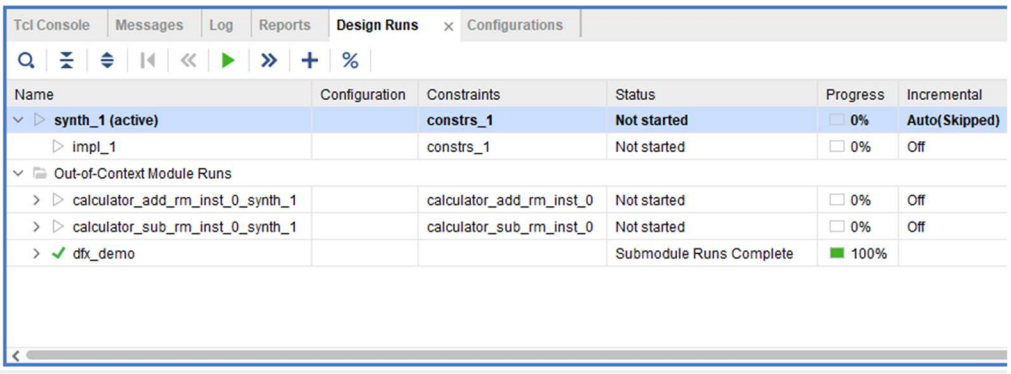
28. **Validate** then **Save** the `calculator_sub_rm` block design.
29. **Validate** then **Save** the `dfx_demo` block design.

Generate targets for the top block design

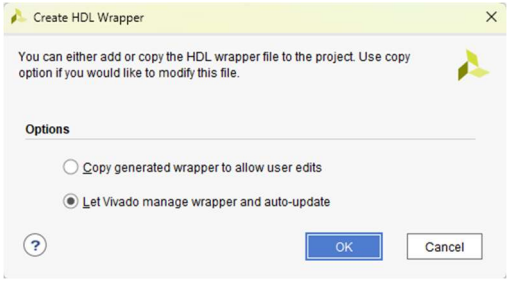
30. In the Sources window, right-click on the `dfx_demo` block design, select **Generate Output Products**, then in the Generate Output Products dialog box select **Generate**.



This creates the synthesizable output products for each IP in `dfx_demo`, building out-of-context synthesis runs for each IP. Under the Design Runs tab, you will see the list of synthesis runs for all the IP contained in static region of `dfx_demo`.



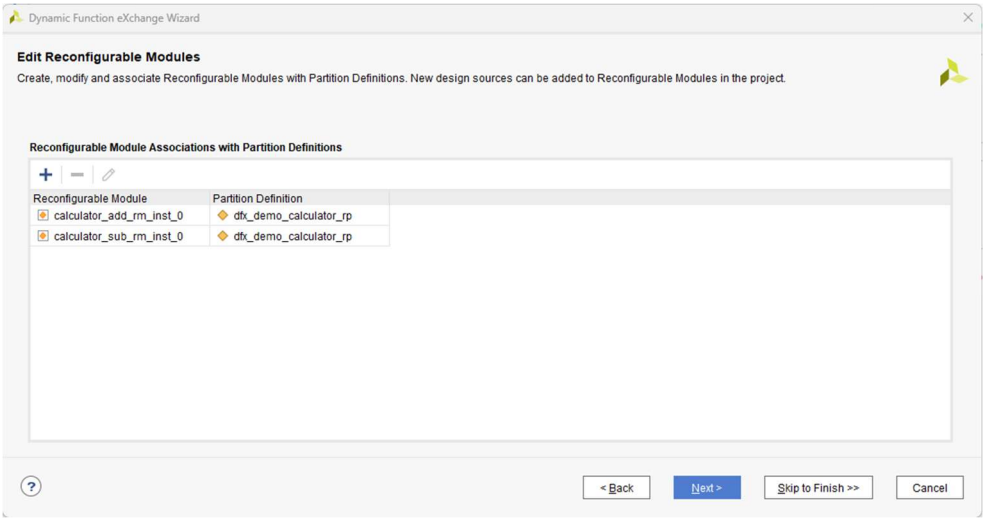
31. Right-click on the `dfx_demo` block design again and select **Create HDL Wrapper***. In the subsequent dialog box, keep **Let Vivado manage wrapper and auto-update** selected then click OK.



Run the DFX Wizard to define the configurations

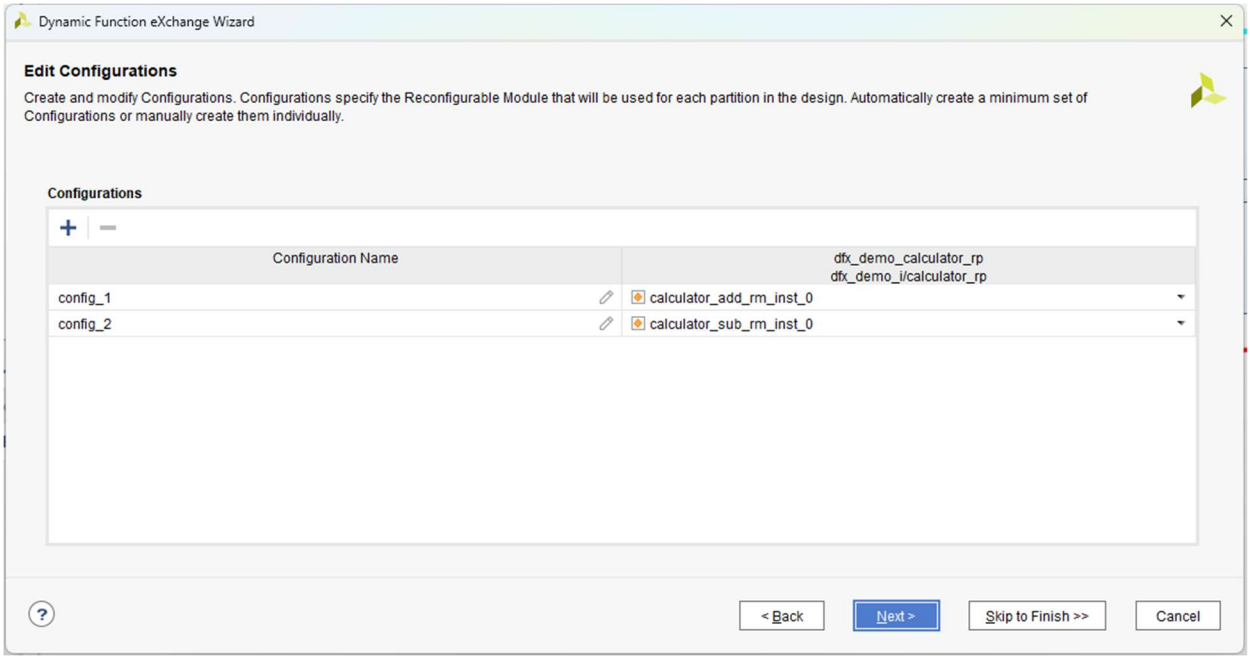
The Dynamic Function eXchange (DFX) Wizard is used to the define relationships between the different parts of the DFX design. Using Block Design Containers, you have created a level of hierarchy for this design that has more than one variant. In a DFX design, the BDC represents a Reconfigurable Partition (RP), and each design source in a BDC (`calculator_add_rm` and `calculator_sub_rm`) is a Reconfigurable Module (RM).

32. Start the DFX Wizard by either clicking on **Dynamic Function eXchange Wizard** in the Flow Navigator, or by selecting that option from the **Tools** menu.
33. When the DFX Wizard starts, click **Next** to see the Edit Reconfigurable Modules step. You will see two RMs, `calculator_add_rm_inst_0` and `calculator_sub_rm _inst_0` that were created earlier.



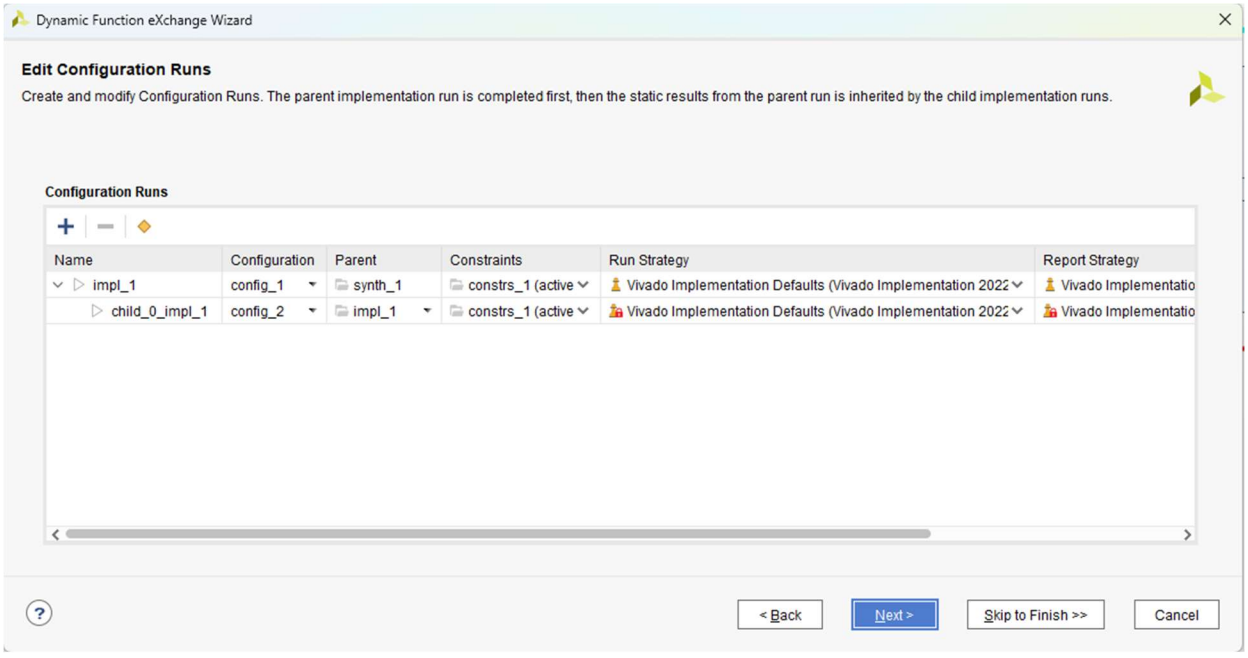
Note that the Block Design Container flow differs from the RTL DFX flow in that you cannot add new variants from within the DFX Wizard. With the Block Design Container DFX flow new variants must be entered from the canvas as you did when you created `calculator_sub_rm`, or by adding new block designs to the project and associating them with the Block Design Container.

34. Click **Next**, and in the Edit Configurations tab, click the **automatically create configurations** link to generate two configurations.



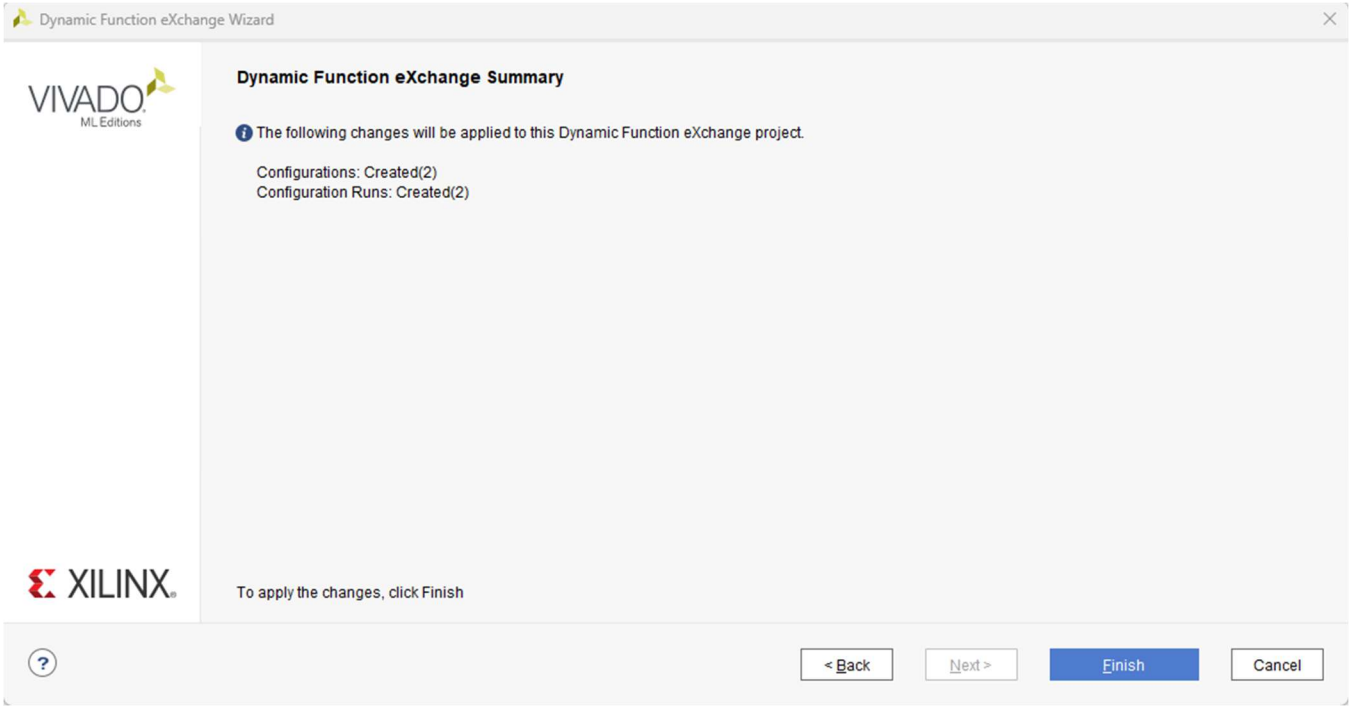
You can create configurations by using the + button. But for designs with a single RP, automatic creation is the easiest way to create configurations covering all RMs. Note that you can edit the configuration name(s) if desired.

35. Click **Next**, and in the Edit Configuration Runs step, click on the **automatically create configuration run** link to create one run per configuration.



In the Edit Configuration Runs tab, you can see the parent-child relationship under the Parent category. A Parent implementation starts with a synthesis run. Then each child run must reference the parent implementation. In this design the parent of `child_0_impl_1` is `impl_1`, and the indentation of the child run illustrates this relationship.

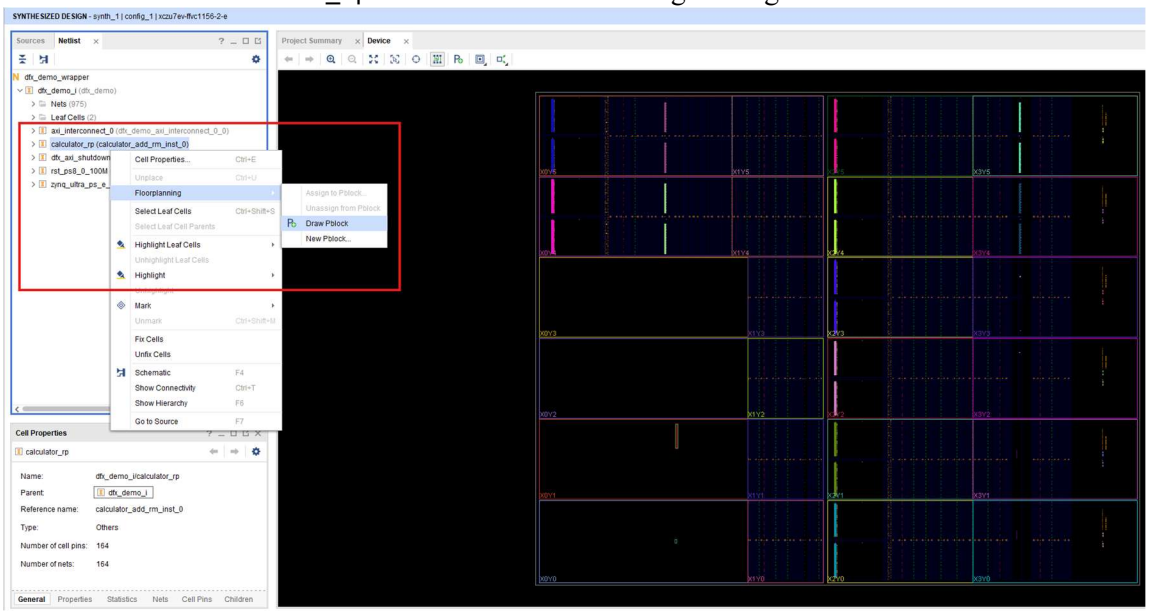
36. Click **Next** then **Finish** to complete this section.



Floorplan the Reconfigurable Partition

All DFX designs require a floorplan, with each RM requiring a pblock containing enough resources to implement any RM that may be used in that partition. In **Vivado DFX / Partial Reconfiguration** design, there are mainly **3 practical ways** to do the floorplan.

- GUI floorplanning
  - Run Synthesis then open Synthesized Design
  - Select `calculator_rp` → Draw Pblock → Assign enough CLB/BRAM/DSP resources



- XDC constraint file
- Tcl script floorplanning

In this design, the XDC pblock constraints have been created for your convenience.

37. In the Sources window, click the + sign to open the Add Source dialog box. Select **Add or create constraints** then **Next**. Click the **Add Files** and navigate to the cloned Github directory `dfx-demo\vivado\constraints` to find and add `dfx_pblock.xdc`. Click **Finish** to add this constraint file to the project.

Now all design sources have been added to the project, and all the settings for a DFX design have been completed. It is time to implement and generate the bitstreams.

38. Under the Flow Navigator, click on the **Generate Bitstream** option. Click **Yes** then **OK** in the resulting dialog boxes to begin **Synthesis** and **Implementation** of the design.
    Typical output locations:

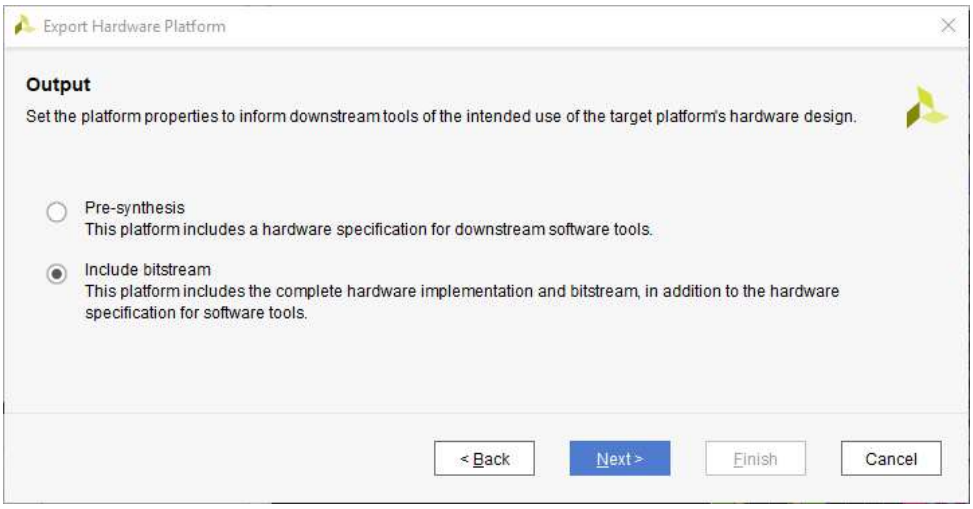
```
project-name.runs/impl_1/
        dfx_demo_wrapper.bit
        dfx_demo_i_calculator_rp_calculator_add_rm_inst_0_partial.bit

project-name.runs/child_0_impl_1/
        dfx_demo_wrapper.bit
        dfx_demo_i_calculator_rp_calculator_sub_rm_inst_0_partial.bit
```

39. Collect the partial bitstreams. Copy them to the SD card and to `ADD.BIT` and `SUB.BIT`

Export the XSA for Vitis application development

40. **Open** the implemented parent design run.
41. Export the hardware by selecting **File > Export > Export Hardware**. Select **Next**, then select **Include bitstream**. Select **Next**, leave the default name and directory, then select **Finish**.



7. Section 3: Vitis Application Flow

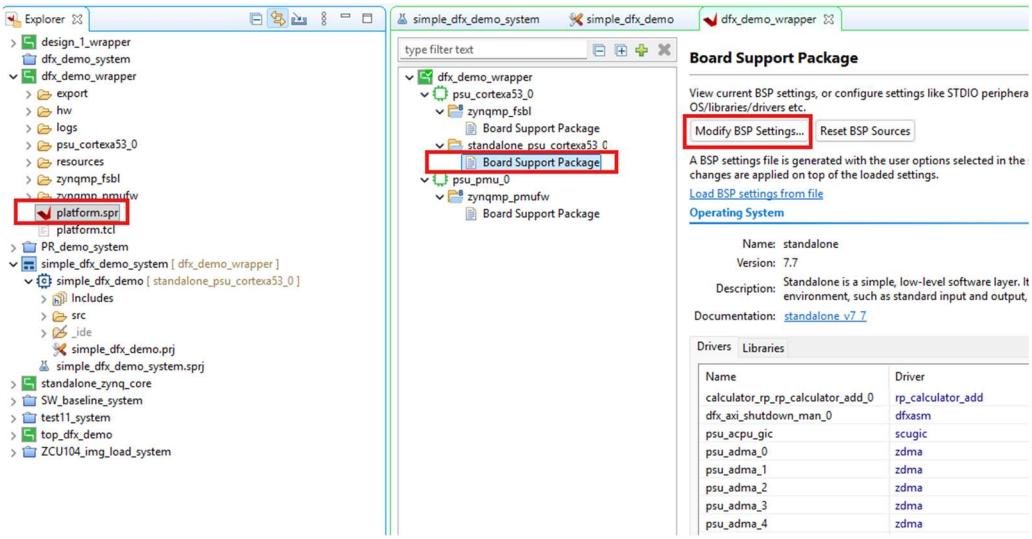
The runtime methodology is the most important part of a robust SoC DFX system. The PS must isolate the RP AXI interface before loading a partial bitstream and reconnect it only after configuration is complete.

The runtime sequence of this demo is presented below.

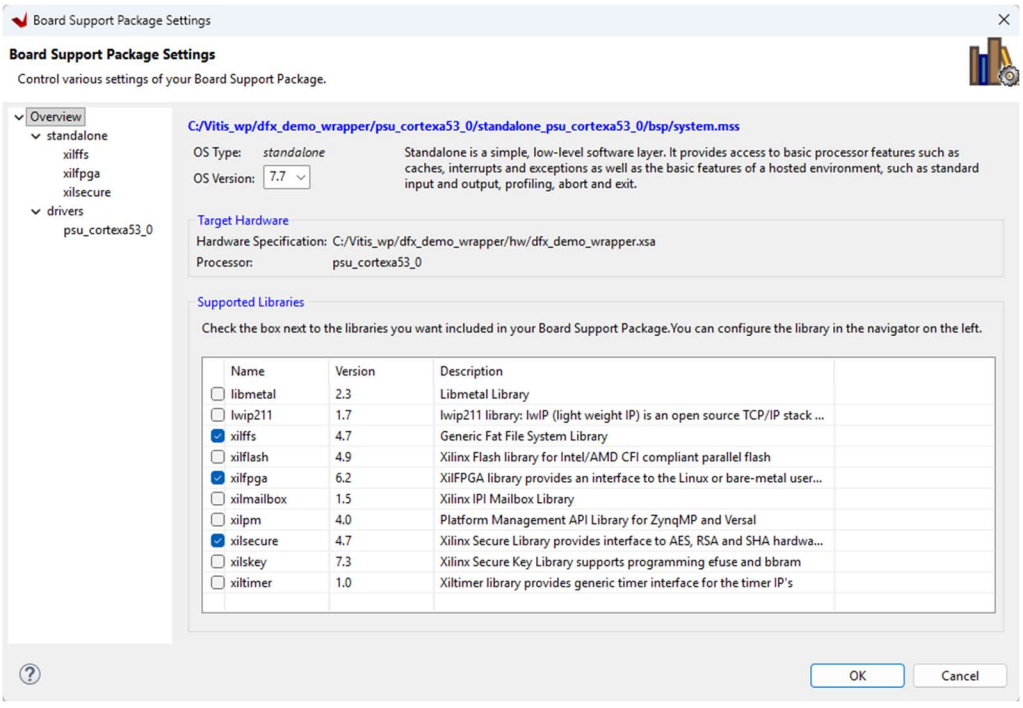
42. Follow section 6 of this guild https://github.com/vandinhtranengg/zybo_memcopy_accel_interrupt for the detailed steps to create a Vitis Application Project in case you not familiar with the procedure.

Enable Required BSP Drivers

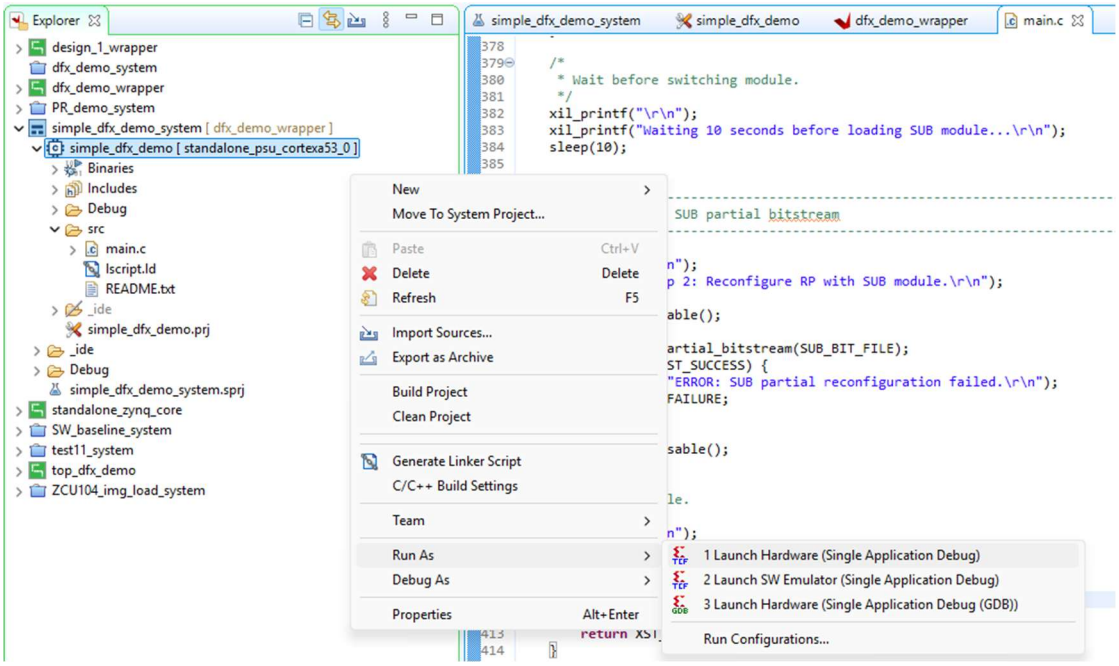
43. Open the BSP Setting, In the Platform Project(`dfx_demo_wrapper`) → `platform.spr` → Board Support Package → Modify BSP Settings



44. Enable these Libraries.

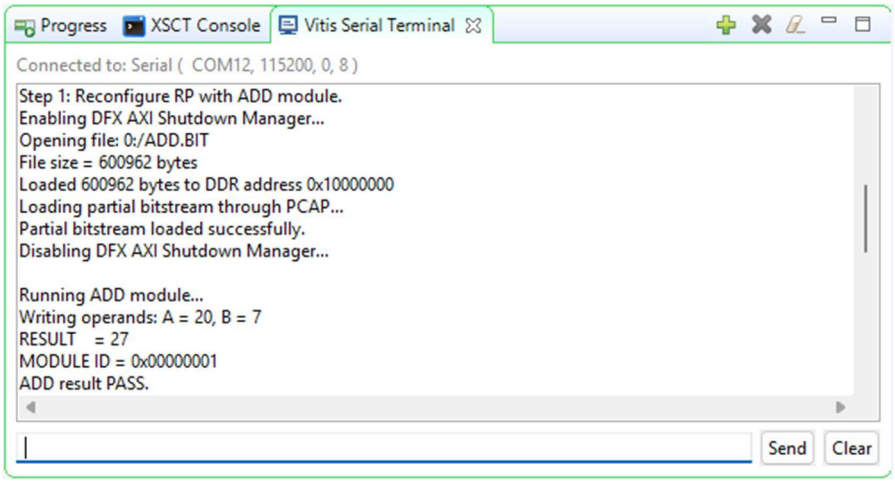


45. Import or copy the `main.c` file to the project `src` directory.
46. Build and then run the application.



Experimental Validation

Check the Vitis Serial Terminal for the result.



8. References

1. Memcopy Accelerator GitHub Repository
2. AMD Vitis HLS User Guide (UG1399) – Getting Started with Vitis HLS
3. AMD Product Guide PG377 – DFX AXI Shutdown Manager
4. AMD Vivado Design Suite Tcl Command Reference Guide (UG835)
5. AMD Vivado Design Suite User Guide: Dynamic Function eXchange (UG909)
6. AMD Vivado Design Suite Tutorial: Dynamic Function eXchange (UG947)
7. Xilinx Vivado Design Tutorials – DFX Decoupler Tutorial
8. PYNQ Forum Discussion – PYNQ 2.7 DFX Partial Reconfiguration under Vivado 2020.2